Informe técnico: primera entrega

Proyecto de Análisis y Diseño de Algoritmos 2026-B, Grupo 3 (Ruta)

Alejandro Duque Muñoz Carlos Felipe Lema Ríos
Universidad de Caldas

6 de noviembre de 2026

1 Introducción

Este informe formaliza los dos problemas del proyecto y demuestra que ambos son difíciles.

- **Parte 1: aceleración de una cadena de etapas con presupuesto.** Se decide qué etapas acelerar, sin pasar de un presupuesto, para que la cadena termine lo antes posible.
- **Parte 2: cobertura mínima de auditoría.** Se escoge el menor número de interacciones que hay que auditar para que todas las categorías de riesgo queden revisadas.

Para cada parte se fijan la entrada, la salida, las restricciones y el objetivo. Después se define la versión de decisión del problema, se demuestra que está en NP y se demuestra que es NP-difícil mediante una reducción polinomial desde un problema NP-completo conocido. En las dos partes la reducción va *desde* el problema conocido *hacia* el nuestro.

Este informe corresponde a la versión `entrega1-1.1` del repositorio, que contiene el código, los casos y los resultados citados en la Sección 4.

Notación. $[n] = \{1, 2, \ldots, n\}$. Se escribe $A \leq_p B$ cuando el problema A se reduce en tiempo polinomial al problema B: si B se resolviera en tiempo polinomial, A también.

2 Parte 1: aceleración de una cadena con presupuesto

2.1 Formalización

Escenario. Un proceso es una cadena serial de n etapas: la etapa $i+1$ empieza cuando termina la etapa i. La etapa i dura t_i. Se puede acelerar pagando un costo c_i, y entonces su duración baja en d_i. Hay un presupuesto total B. Se quiere decidir qué etapas acelerar para que la duración total sea la menor posible sin gastar más de B.

Entrada.

- Un entero $n \geq 1$.
- Enteros $t_i \geq 0$, $c_i \geq 0$ y d_i con $0 \leq d_i \leq t_i$, para cada $i \in [n]$.
- Un presupuesto entero $B \geq 0$.

Salida. Un vector $x \in \{0,1\}^n$, donde $x_i = 1$ significa que se acelera la etapa i.

Duración final y costo total.

$$T_{\text{fin}}(x) = \sum_{i=1}^{n} (t_i - d_i\, x_i), \qquad C_{\text{tot}}(x) = \sum_{i=1}^{n} c_i\, x_i.$$

Problema.

$$\min\ T_{\text{fin}}(x) \quad \text{sujeto a} \quad C_{\text{tot}}(x) \le B, \quad x \in \{0,1\}^n.$$

El vector $x = (0, \ldots, 0)$ siempre es factible (cuesta $0 \le B$), así que siempre existe una solución óptima.

Equivalencia con maximizar el ahorro. Como la cadena es serial, todas las etapas están en el camino crítico y las duraciones se suman. Entonces

$$T_{\text{fin}}(x) = \sum_{i=1}^{n} t_i - \sum_{i=1}^{n} d_i\, x_i.$$

La suma $\sum_i t_i$ no depende de x. Por lo tanto, minimizar T_{fin} es lo mismo que maximizar el ahorro $D(x) = \sum_i d_i x_i$:

$$\max\ D(x) = \sum_{i=1}^{n} d_i\, x_i \quad \text{sujeto a} \quad \sum_{i=1}^{n} c_i\, x_i \le B, \quad x \in \{0,1\}^n.$$

Este es el problema de la mochila 0/1 con pesos c_i, valores d_i y capacidad B.

Tamaño de la entrada. Una instancia son $3n$ enteros más B. Los parámetros que controlan el costo de los algoritmos son n y B. Para escribir B en binario bastan $k = \lfloor \log_2 B \rfloor + 1$ bits, es decir, B es del orden de 2^k. Por eso B puede ser *exponencialmente grande* respecto de la longitud de su escritura. Esto se usa en la Sección 2.4.

Flujos con rutas paralelas. Los casos de prueba pueden traer un flujo que no sea una cadena (más etapas, estaciones o caminos). En ese caso la duración total es la del camino más largo y no la suma de todas las duraciones. La cadena es el caso particular que se desarrolla en las Secciones 2.1 a 2.4, y el caso general se trata en la Sección 2.5.

Ejemplo 2.1 (cuatro etapas)**.** Sean $n = 4$, $B = 7$ y

$$t = (10, 8, 12, 6), \qquad c = (4, 3, 5, 2), \qquad d = (5, 4, 7, 3).$$

Sin acelerar nada, $\sum_i t_i = 36$. Hay $2^4 = 16$ vectores posibles; la Tabla 1 los recorre todos.

Hay 9 vectores factibles. El mejor es $x = (0, 0, 1, 1)$: cuesta $5 + 2 = 7 \le B$, ahorra $7 + 3 = 10$ y deja la cadena en $T_{\text{fin}} = 36 - 10 = 26$. Es el único óptimo. Acelerar las etapas 1 y 2 también gasta todo el presupuesto, pero ahorra solo 9.

2.2 Versión de decisión y pertenencia a NP

Definición 2.2 (ACE-DEC)**.** Dada una instancia (n, t, c, d, B) y un entero $K \ge 0$, ¿existe $x \in \{0,1\}^n$ con $C_{\text{tot}}(x) \le B$ y $D(x) \ge K$?

Por la igualdad $T_{\text{fin}}(x) = \sum_i t_i - D(x)$, la condición $D(x) \ge K$ equivale a $T_{\text{fin}}(x) \le \sum_i t_i - K$. Es decir, ACE-DEC pregunta si se puede lograr una duración final de a lo sumo $\sum_i t_i - K$ con el presupuesto dado.

Si se supiera resolver la versión de optimización, se resolvería ACE-DEC comparando el ahorro óptimo con K. Por eso la versión de decisión nunca es más difícil que la de optimización.

x	$C_{\text{tot}}(x)$	$D(x)$	$T_{\text{fin}}(x)$
$(0,0,0,0)$	0	0	36
$(0,0,0,1)$	2	3	33
$(0,0,1,0)$	5	7	29
$\mathbf{(0,0,1,1)}$	**7**	**10**	**26**
$(0,1,0,0)$	3	4	32
$(0,1,0,1)$	5	7	29
$(0,1,1,0)$	8	–	–
$(0,1,1,1)$	10	–	–
$(1,0,0,0)$	4	5	31
$(1,0,0,1)$	6	8	28
$(1,0,1,0)$	9	–	–
$(1,0,1,1)$	11	–	–
$(1,1,0,0)$	7	9	27
$(1,1,0,1)$	9	–	–
$(1,1,1,0)$	12	–	–
$(1,1,1,1)$	14	–	–

Tabla 1: Los 16 vectores del Ejemplo 2.1. Un guion indica que el vector no es factible ($C_{\text{tot}} > B$).

Lema 2.3. ACE-DEC $\in$ NP.

Demostración. El certificado es el vector $x \in \{0,1\}^n$, que ocupa n bits. Para verificarlo se calculan $C_{\text{tot}}(x)$ y $D(x)$ con un recorrido de las n etapas y se comprueba $C_{\text{tot}}(x) \le B$ y $D(x) \ge K$. Cuesta $O(n)$, que es polinomial. □

2.3 NP-dureza: reducción desde la mochila 0/1

Definición 2.4 (KNAP-DEC)**.** Dados m objetos con pesos enteros positivos $w_1, \ldots, w_m$ y valores enteros positivos $v_1, \ldots, v_m$, una capacidad entera W y un valor objetivo entero V: ¿existe $A \subseteq [m]$ con $\sum_{i \in A} w_i \le W$ y $\sum_{i \in A} v_i \ge V$?

KNAP-DEC es NP-completo (Karp, 1972; Garey y Johnson, 1979).

Construcción f. A partir de una instancia (w, v, W, V) de KNAP-DEC se construye la instancia de ACE-DEC con

$$n = m, \qquad c_i = w_i, \qquad d_i = v_i, \qquad t_i = v_i + 1 \quad (i \in [m]), \qquad B = W, \qquad K = V.$$

Los números se copian tal cual. La duración $t_i = v_i + 1$ se elige para cumplir $d_i \le t_i$ (de hecho $d_i < t_i$, así que ninguna etapa llega a duración 0); cualquier $t_i \ge v_i$ serviría.

Tiempo de la construcción. Se escriben $3m + 2$ números copiando o sumando 1 a lo que ya hay. Cuesta $O(m)$, que es polinomial.

Teorema 2.5. KNAP-DEC $\le_p$ ACE-DEC. Más precisamente, (w, v, W, V) es una instancia afirmativa de KNAP-DEC si y solo si $f(w, v, W, V)$ es una instancia afirmativa de ACE-DEC.

Demostración. ($\Rightarrow$) Sea $A \subseteq [m]$ con $\sum_{i \in A} w_i \le W$ y $\sum_{i \in A} v_i \ge V$. Se define $x_i = 1$ si $i \in A$ y $x_i = 0$ si no. Entonces

$$C_{\text{tot}}(x) = \sum_{i \in A} c_i = \sum_{i \in A} w_i \le W = B, \qquad D(x) = \sum_{i \in A} d_i = \sum_{i \in A} v_i \ge V = K.$$

Luego x es una solución afirmativa de ACE-DEC.

(⇐) Sea $x \in \{0,1\}^m$ con $C_{\text{tot}}(x) \le B$ y $D(x) \ge K$. Se define $A = \{i : x_i = 1\}$. Entonces

$$\sum_{i \in A} w_i = C_{\text{tot}}(x) \le B = W, \qquad \sum_{i \in A} v_i = D(x) \ge K = V.$$

Luego A es una solución afirmativa de KNAP-DEC. □

Corolario 2.6. ACE-DEC es NP-completo, y el problema de optimización de la Parte 1 es NP-difícil.

Demostración. ACE-DEC $\in$ NP por el lema anterior y es NP-difícil por el Teorema 2.5. Si el problema de optimización se resolviera en tiempo polinomial, ACE-DEC también (comparando el ahorro óptimo con K), y entonces NP = P. □

Observación 2.7. La reducción produce siempre cadenas. Como la cadena es un caso particular de un flujo cualquiera (Sección 2.5), el problema con flujos generales contiene al de la cadena y también es NP-difícil.

Observación 2.8. La reducción copia los números sin agrandarlos. La dificultad viene de que B puede ser exponencial respecto de su escritura. Por eso no hay contradicción con un algoritmo de tiempo $O(n \cdot B)$: es polinomial en el *valor* de B y no en su cantidad de bits (algoritmo pseudopolinomial, Sección 2.4).

Ejemplo 2.9 (la reducción con tres objetos)**.** Sea la instancia de KNAP-DEC con $w = (2,3,4)$, $v = (3,4,5)$, $W = 5$ y $V = 7$. La construcción da

$$t = (4,5,6), \quad c = (2,3,4), \quad d = (3,4,5), \quad B = 5, \quad K = 7.$$

El conjunto $A = \{1,2\}$ pesa $2+3 = 5 \le 5$ y vale $3+4 = 7 \ge 7$, así que la respuesta de KNAP-DEC es SÍ. El vector $x = (1,1,0)$ cuesta $5 \le B$ y ahorra $7 \ge K$; deja la cadena en $T_{\text{fin}} = 15 - 7 = 8 = \sum_i t_i - K$. La respuesta de ACE-DEC también es SÍ.

2.4 Algoritmo exacto para la cadena y su eficiencia

Cuando el flujo es una cadena el problema es la mochila 0/1, y se resuelve de forma exacta con programación dinámica. Se numeran las etapas en el orden de la cadena.

Subestructura. Para $0 \le i \le n$ y $0 \le b \le B$ sea

$$DP[i][b] = \max\Big\{ \sum_{j=1}^{i} d_j\, x_j \;:\; \sum_{j=1}^{i} c_j\, x_j \le b,\; x \in \{0,1\}^i \Big\},$$

el mayor ahorro que se logra usando solo las primeras i etapas con presupuesto b. La respuesta es $DP[n][B]$.

Recurrencia.

$$DP[0][b] = 0, \qquad DP[i][b] = \begin{cases} DP[i-1][b] & \text{si } c_i > b, \\ \max\big(DP[i-1][b],\; DP[i-1][b-c_i] + d_i\big) & \text{si } c_i \le b. \end{cases}$$

Lema 2.10. La recurrencia calcula $DP[i][b]$ correctamente para todo i y b.

Demostración. Por inducción sobre i. Para $i = 0$ no hay etapas y el ahorro máximo es 0. Sea $i \geq 1$ y una solución óptima x para (i, b). Si $x_i = 0$, usa solo las etapas $1, \ldots, i-1$ con presupuesto b, y por hipótesis su ahorro es a lo sumo $DP[i-1][b]$. Si $x_i = 1$, entonces $c_i \leq b$ y las demás etapas usan presupuesto a lo sumo $b - c_i$, así que su ahorro es a lo sumo $DP[i-1][b-c_i]$ y el total es a lo sumo $DP[i-1][b-c_i] + d_i$. Luego $DP[i][b]$ es a lo sumo el máximo de las dos opciones. Recíprocamente, cada opción se realiza con una solución factible. Por lo tanto vale la igualdad. □

Reconstrucción. Junto con $DP[i][b]$ se guarda $toma[i][b] = 1$ solo cuando acelerar la etapa i mejora *estrictamente* el ahorro. Después se recorre la cadena desde $i = n$ con $b = B$: si $toma[i][b] = 1$ se acelera la etapa i y b baja en c_i. El desempate es fijo: ante un empate no se acelera. La solución reconstruida tiene ahorro $DP[n][B]$ y costo a lo sumo B.

Algoritmo 1 Programación dinámica para la cadena. Entrada: n, t, c, d (en el orden de la cadena) y B. Salida: $x \in \{0,1\}^n$ y el ahorro máximo.

```
1  para b ← 0 hasta B
2    DP[0][b] ← 0
3  para i ← 1 hasta n
4    para b ← 0 hasta B
5      DP[i][b] ← DP[i − 1][b];   toma[i][b] ← 0
6      si c_i ≤ b
7        si DP[i − 1][b − c_i] + d_i > DP[i − 1][b]
8          DP[i][b] ← DP[i − 1][b − c_i] + d_i;   toma[i][b] ← 1
9  b ← B
10 para i ← n hasta 1
11   si toma[i][b] = 1
12     x_i ← 1;   b ← b − c_i
13   si no x_i ← 0
14 devolver x y DP[n][B]
```

Conteo de instrucciones por línea

Cada línea L_k del Algoritmo 1 cuesta una constante C_k por ejecución. Un ciclo **para** que recorre m valores evalúa su encabezado $m + 1$ veces. Sea

$$S = \sum_{i=1}^{n} \max(0,\, B - c_i + 1)$$

el número de veces que la condición de la línea 6 es verdadera (para cada i, los valores b con $c_i \leq b \leq B$), $M \leq S$ el número de veces que la línea 7 mejora el ahorro, y k el número de etapas aceleradas.

Línea	Instrucción	Costo	Veces
1	**para** $b \leftarrow 0$ **hasta** B	C_1	$B+2$
2	$DP[0][b] \leftarrow 0$	C_2	$B+1$
3	**para** $i \leftarrow 1$ **hasta** n	C_3	$n+1$
4	**para** $b \leftarrow 0$ **hasta** B	C_4	$\sum_{i=1}^{n}(B+2) = n(B+2)$
5	copiar $DP[i-1][b]$ y $toma \leftarrow 0$	C_5	$\sum_{i=1}^{n}(B+1) = n(B+1)$
6	**si** $c_i \le b$	C_6	$n(B+1)$
7	**si** $DP[i-1][b-c_i] + d_i > DP[i-1][b]$	C_7	$S \le n(B+1)$
8	actualizar $DP[i][b]$ y $toma \leftarrow 1$	C_8	$M \le S$
9	$b \leftarrow B$	C_9	1
10	**para** $i \leftarrow n$ **hasta** 1	C_{10}	$n+1$
11	**si** $toma[i][b] = 1$	C_{11}	n
12	$x_i \leftarrow 1$; $b \leftarrow b - c_i$	C_{12}	k
13	$x_i \leftarrow 0$	C_{13}	$n-k$
14	**devolver**	C_{14}	1

Sumando costo por veces:

$$T(n,B) = C_1(B+2) + C_2(B+1) + C_3(n+1) + (C_4+C_5+C_6)\,n(B+1) + C_4\,n \\ + C_7\,S + C_8\,M + C_9 + C_{10}(n+1) + C_{11}\,n + C_{12}\,k + C_{13}(n-k) + C_{14}.$$

Con $M \le S \le n(B+1)$ y $C'_{12} = \max(C_{12}, C_{13})$, agrupando por potencias de n y B:

$$T(n,B) \;\le\; a\,nB + b\,n + c\,B + e,$$

donde

$$a = C_4 + C_5 + C_6 + C_7 + C_8,$$
$$b = C_3 + 2C_4 + C_5 + C_6 + C_7 + C_8 + C_{10} + C_{11} + C'_{12},$$
$$c = C_1 + C_2,$$
$$e = 2C_1 + C_2 + C_3 + C_9 + C_{10} + C_{14}.$$

Como además $T(n,B) \ge (C_4+C_5+C_6)\,nB$ en cualquier caso, la función de eficiencia es

$$\boxed{T(n,B) = \Theta(n \cdot B)}$$

Peor caso: S y M alcanzan su máximo $n(B+1)$ (por ejemplo, todas las etapas con costo $c_i = 0$ y ahorro $d_i > 0$). *Mejor caso:* ninguna etapa cabe ($c_i > B$ para todo i, así que $S = M = 0$). En ambos la operación que más se repite son las comparaciones de las líneas 4 a 6, que se ejecutan $n(B+1)$ veces, y por eso el orden es el mismo.

Memoria. La tabla tiene $(n+1)(B+1)$ celdas y la reconstrucción necesita los $n(B+1)$ valores de *toma*, así que el espacio es $O(n \cdot B)$.

Preparación. Ordenar las etapas según el flujo y comprobar que es una cadena cuesta $O(n+m)$, con $m = n-1$ aristas; queda absorbido por nB.

Por qué es pseudopolinomial

El tiempo $\Theta(nB)$ es polinomial en el *valor* de B, pero no en el tamaño de la entrada. Escribir B en binario ocupa $k = \lfloor \log_2 B \rfloor + 1$ bits, así que $B \approx 2^k$ y $n \cdot B \approx n\,2^k$. Agregar un solo bit a la escritura de B duplica el trabajo. Por ejemplo, con $B = 2^{40}$ la entrada necesita solo 41 bits para B, pero la tabla tendría del orden de $n \cdot 10^{12}$ celdas. Un algoritmo así se llama pseudopolinomial. No contradice la NP-dureza: la mochila es NP-difícil en el sentido débil, justamente porque su dificultad desaparece cuando B es pequeño en valor.

2.5 Flujos con rutas paralelas

Formalización. El flujo es un grafo dirigido acíclico $G = (V, E)$ cuyos nodos son las etapas y cuyas aristas $(u, v) \in E$ indican que v no empieza hasta que termina u. Con $x \in \{0, 1\}^n$ cada etapa pesa $w_i(x) = t_i - d_i\, x_i$, y la duración total es la del camino más largo:

$$T_{\text{fin}}(x) = \max_{P \text{ camino de } G} \sum_{i \in P} (t_i - d_i\, x_i), \qquad C_{\text{tot}}(x) = \sum_{i=1}^{n} c_i\, x_i \le B.$$

Se minimiza $T_{\text{fin}}(x)$. Cuando G es una cadena, el único camino maximal contiene todas las etapas y se recupera la Sección 2.4. Aquí *ya no* es cierto que minimizar la duración sea maximizar $\sum d_i x_i$: acelerar una etapa fuera del camino crítico no reduce nada.

Dificultad. La versión de decisión (¿existe x con $C_{\text{tot}}(x) \le B$ y $T_{\text{fin}}(x) \le \tau$?) está en NP: el certificado es x y el camino más largo de un grafo acíclico se calcula en $O(n + m)$. Es NP-difícil porque la cadena es un caso particular, y la reducción de la Sección 2.3 solo usa cadenas.

Algoritmo exacto (ramificación y acotamiento). Se consideran *candidatas* las etapas con $d_i > 0$ y $c_i \le B$; las demás nunca se aceleran. Se decide etapa por etapa (primero las de mayor ahorro d_i) si se acelera o no, y en cada nodo del árbol de decisiones se calculan dos cotas inferiores de la duración final que se puede alcanzar con lo decidido hasta ahora:

1. *Cota 1:* se aceleran gratis todas las candidatas aún no decididas y se calcula el camino más largo.
2. *Cota 2:* se toma el camino P más largo cuando solo cuentan las aceleraciones ya decididas, y se le resta lo máximo que una mochila *fraccionaria* permitiría ahorrar sobre las candidatas sin decidir de P con el presupuesto que queda (parte entera).

Si alguna cota es mayor o igual que la mejor duración encontrada, se poda el nodo. La mejor solución parte de un voraz inicial (acelerar sobre el camino crítico por razón d_i/c_i) y mejora cada vez que un nodo da una duración menor.

Lema 2.11. Las dos cotas son cotas inferiores válidas: toda solución factible que respete las decisiones del nodo tiene duración al menos igual a cada una.

Demostración. Cota 1. Cualquier completación acelera un subconjunto de las candidatas sin decidir, así que ninguna etapa pesa menos que en la configuración "todo acelerado gratis". El camino más largo es monótono en los pesos. *Cota 2.* Una completación x' cumple $T_{\text{fin}}(x') \ge \sum_{i \in P} w_i(x')$, que es la longitud de P con lo decidido menos $\sum_{i \in P \text{ sin decidir}} d_i x'_i$. Esa suma es a lo sumo el óptimo fraccionario de la mochila sobre esas etapas con presupuesto $B -$ (costo ya gastado), y como el ahorro real es entero, a lo sumo su parte entera. □

Si la búsqueda termina sin alcanzar el tope de nodos, la duración devuelta es óptima: cada nodo podado no contiene ninguna solución estrictamente mejor. Si se alcanza el tope, el programa devuelve la mejor solución encontrada y lo avisa con `optimo_probado = false`.

Eficiencia. Cada nodo calcula caminos más largos en $O(n + m)$ y ordena a lo sumo n etapas, así que cuesta $O(n + m + n \log n)$. El árbol tiene a lo sumo $2^{n'+1} - 1$ nodos, con n' el número de candidatas:

$$T = O\big(2^{n'}\,(n + m + n \log n)\big).$$

Es exponencial en el peor caso, como corresponde a un problema NP-difícil. En la práctica las cotas podan casi todo: la Tabla 5 muestra flujos de 20 y 30 etapas resueltos en milisegundos y un caso exigente de 40 etapas por capas de ancho 8 resuelto en cerca de un segundo.

El programa decide solo qué método usar: si el flujo es una cadena aplica el Algoritmo 1; si no, ramificación y acotamiento.

3 Parte 2: cobertura mínima de auditoría

3.1 Formalización

Escenario. Hay n interacciones candidatas a auditar y r categorías de riesgo (sesgo, fuga de datos personales, fallas de formato, etc.). Auditar una interacción revela todas las categorías de riesgo que contiene. Se quiere auditar el menor número posible de interacciones de modo que cada categoría quede revisada al menos una vez. En los archivos de entrada cada interacción es una etapa del flujo y S_j es la lista de riesgos que cubre esa etapa.

Entrada.

- Un entero $r \geq 1$ y el universo de categorías $U = \{1, 2, \ldots, r\}$.
- Una familia $\mathcal{S} = (S_1, S_2, \ldots, S_n)$, donde $S_j \subseteq U$ es el conjunto de categorías que cubre la interacción j.

Salida. Un conjunto de índices $J \subseteq [n]$ (las interacciones a auditar). De forma equivalente, un vector $y \in \{0,1\}^n$ con $y_j = 1 \iff j \in J$.

Restricción. Cada categoría queda cubierta:

$$\bigcup_{j \in J} S_j = U \qquad \Longleftrightarrow \qquad \forall k \in U : \sum_{j \,:\, k \in S_j} y_j \geq 1.$$

Objetivo. Minimizar $|J| = \sum_{j=1}^{n} y_j$.

Condición de factibilidad. Existe al menos una solución si y solo si $\bigcup_{j=1}^{n} S_j = U$. Si se cumple, $J = [n]$ ya es una solución, así que existe un óptimo. Si no se cumple, alguna categoría k no aparece en ningún S_j y ninguna auditoría la cubre. Esta condición se comprueba antes de correr cualquier algoritmo: se marca en un arreglo de r posiciones cada categoría que aparece en algún S_j y después se listan las no marcadas. Cuesta $O(r + s)$ con $s = \sum_j |S_j| \leq n \cdot r$, es decir, $O(n \cdot r)$.

Tamaño de la entrada: (n, r). Una instancia son n subconjuntos de un universo de r elementos. Escrita como matriz de incidencia de $n \times r$ bits (la fila j, columna k vale 1 si $k \in S_j$), ocupa exactamente $n \cdot r$ bits. Escrita como listas, ocupa $\Theta(s \log r)$ bits con $s \leq n \cdot r$. En ambas representaciones lo que crece cuando el problema se hace más grande son n (más interacciones para escoger) y r (más riesgos que cubrir). A diferencia de la Parte 1, aquí ningún número tiene un *valor* que controle el tiempo: los números de la entrada son índices entre 1 y r. Por eso las cotas polinomiales en n y r son polinomiales en el tamaño de la entrada.

Se usa también $d = \max_j |S_j| \leq r$, el máximo de categorías que cubre una sola interacción. Aparece en la cota del algoritmo voraz.

Definición 3.1 (AUD-DEC)**.** Dados r, $S_1, \ldots, S_n$ y un entero $k \geq 0$: ¿existe $J \subseteq [n]$ con $|J| \leq k$ y $\bigcup_{j \in J} S_j = U$?

Lema 3.2. AUD-DEC $\in$ NP.

Demostración. El certificado es el conjunto J. Para verificarlo se comprueba $|J| \leq k$ en $O(n)$ y se marcan en un arreglo de r posiciones las categorías de $\bigcup_{j \in J} S_j$, en $O(n \cdot r)$. Después se revisa que las r posiciones estén marcadas. En total es polinomial. □

3.2 NP-dureza: reducción desde Cobertura de Vértices

Definición 3.3 (VC-DEC)**.** Dados un grafo no dirigido simple $G = (V, E)$ y un entero $k \geq 0$: ¿existe $C \subseteq V$ con $|C| \leq k$ tal que toda arista tiene al menos un extremo en C?

VC-DEC es NP-completo (Karp, 1972). Se demuestra VC-DEC $\leq_p$ AUD-DEC: la reducción va desde el problema conocido hacia el nuestro.

Construcción $f(G, k)$**.** Sean $V = \{v_1, \ldots, v_{|V|}\}$ y las aristas numeradas $E = \{e_1, \ldots, e_m\}$.

1. *Categorías = aristas:* $r = m$ y $U = \{1, \ldots, m\}$. La categoría ℓ representa la arista e_ℓ.
2. *Interacciones = vértices:* $n = |V|$ y, para cada vértice v, $S_v = \{\ell : v \in e_\ell\}$ (las aristas que tocan a v).
3. *Mismo umbral:* k.

Caso borde. Si $E = \emptyset$, la respuesta de VC-DEC es SÍ para cualquier k. Entonces f devuelve una instancia fija que también es SÍ: $r = 1$, $S_1 = \{1\}$, $k = 1$. Hace falta porque la entrada exige $r \geq 1$.

Tiempo de la construcción. Se crean $|V|$ listas vacías en $O(|V|)$. Luego se recorre cada arista $e_\ell = \{u, v\}$ y se agrega ℓ a S_u y a S_v, con costo $O(1)$ por arista, es decir, $O(|E|)$. En total $O(|V| + |E|)$, que es polinomial.

La instancia construida siempre es factible, porque toda arista tiene un extremo y así $\bigcup_v S_v = U$. Un vértice aislado produce $S_v = \emptyset$, lo cual es válido.

Teorema 3.4. VC-DEC $\leq_p$ AUD-DEC. Más precisamente, G tiene una cobertura de vértices de tamaño $\leq k$ si y solo si $f(G, k)$ tiene una auditoría de tamaño $\leq k$.

Demostración. (Se trata el caso $E \neq \emptyset$; el caso $E = \emptyset$ se resolvió arriba.)

($\Rightarrow$) Sea C una cobertura de vértices con $|C| \leq k$. Se toma $J = C$, así que $|J| \leq k$. Sea $\ell \in U$ cualquiera. La arista e_ℓ tiene un extremo $v \in C$, por ser C cobertura de vértices. Por construcción, $\ell \in S_v$ con $v \in J$. Luego $\bigcup_{v \in J} S_v = U$.

($\Leftarrow$) Sea J una auditoría con $|J| \leq k$ y $\bigcup_{v \in J} S_v = U$. Se toma $C = J$. Sea e_ℓ cualquier arista. La categoría ℓ está cubierta, así que existe $v \in J$ con $\ell \in S_v$. Por construcción, esto significa que $v \in e_\ell$. Luego v es un extremo de e_ℓ y $v \in C$. Toda arista queda cubierta y $|C| \leq k$. □

Corolario 3.5. AUD-DEC es NP-completo, y el problema de optimización de la Parte 2 (hallar el J de tamaño mínimo) es NP-difícil.

Demostración. AUD-DEC $\in$ NP por el lema de la sección anterior y es NP-difícil por el Teorema 3.4. Resolver la optimización permite resolver la decisión comparando $|J^*|$ con k. □

Ejemplo 3.6 (grafo de cinco vértices)**.** Sean $V = \{a, b, c, d, e\}$,
$E = \{e_1{=}ab,\ e_2{=}ac,\ e_3{=}bc,\ e_4{=}bd,\ e_5{=}ce,\ e_6{=}de\}$ y $k = 3$. La construcción da $r = 6$ categorías y $n = 5$ interacciones:

Vértice (interacción)	Aristas que toca (S_v)
a	$\{1, 2\}$
b	$\{1, 3, 4\}$
c	$\{2, 3, 5\}$
d	$\{4, 6\}$
e	$\{5, 6\}$

El conjunto $C = \{b, c, d\}$ es una cobertura de vértices: b cubre ab, bc y bd; c cubre ac y ce; d cubre de. En la instancia de auditoría, $S_b \cup S_c \cup S_d = \{1, 3, 4\} \cup \{2, 3, 5\} \cup \{4, 6\} = \{1, \ldots, 6\}$, también con 3 interacciones.

No hay cobertura de tamaño 2: cada vértice toca a lo sumo 3 aristas, así que dos vértices solo cubren las 6 aristas si ambos tocan 3 y no comparten ninguna. Solo b y c tocan 3 aristas, y comparten la arista bc (juntos cubren 5). Por lo tanto el mínimo es 3 en las dos instancias.

3.3 Algoritmo voraz y cota de aproximación

Idea. Mientras falten categorías por cubrir, elegir la interacción que cubre más categorías todavía no cubiertas.

Desempate (fijo). Si varias interacciones empatan, gana la que aparece primero en la entrada. Así la ejecución es reproducible: la misma entrada siempre produce la misma salida.

Algoritmo 2 Voraz para la cobertura mínima de auditoría. Entrada: r y $S_1, \ldots, S_n$ (instancia factible). Salida: $J \subseteq [n]$ con $\bigcup_{j \in J} S_j = U$.

```
1  para k ← 1 hasta r
2     cubierta[k] ← falso
3  para j ← 1 hasta n
4     elegida[j] ← falso
5  faltan ← r;   J ← ∅
6  mientras faltan > 0
7     mejor ← nulo;   g* ← 0
8     para j ← 1 hasta n
9        si elegida[j]: pasar al siguiente j
10       g ← 0
11       para cada k ∈ S_j
12          si no cubierta[k]: g ← g + 1
13       si g > g*: mejor ← j;   g* ← g
14    elegida[mejor] ← verdadero;   J ← J ∪ {mejor}
15    para cada k ∈ S_mejor
16       si no cubierta[k]
17          cubierta[k] ← verdadero;   faltan ← faltan − 1
18 devolver J
```

Como se usa $g > g^*$ (desigualdad estricta) y j crece, entre interacciones empatadas se queda la primera.

Lema 3.7. Si la instancia es factible, el Algoritmo 2 termina, devuelve una cobertura y hace $p \leq \min(n, r)$ vueltas del ciclo principal.

Demostración. Mientras *faltan* > 0 queda una categoría k sin cubrir. Por factibilidad existe j con $k \in S_j$, y ese j no está elegido (si lo estuviera, k estaría cubierta). Luego en la vuelta se encuentra una interacción con ganancia $g^* \geq 1$: cada vuelta cubre al menos una categoría nueva. Como solo hay r categorías, hay a lo sumo r vueltas. Además, cada vuelta elige una interacción distinta (las elegidas se saltan), así que hay a lo sumo n vueltas. Al terminar, *faltan* $= 0$, es decir, todas las categorías están cubiertas. □

Ejemplo 3.8 (el voraz no siempre es óptimo)**.** Sean $r = 6$ y $S_1 = \{1, 2, 3, 4\}$, $S_2 = \{1, 3, 5\}$, $S_3 = \{2, 4, 6\}$.

Vuelta	Elegida	Categorías nuevas	Faltan
1	S_1	$\{1,2,3,4\}$	2
2	S_2	$\{5\}$	1
3	S_3	$\{6\}$	0

El voraz devuelve $J_g = \{1,2,3\}$, de tamaño 3. El óptimo es $J^* = \{2,3\}$, de tamaño 2, porque $S_2 \cup S_3 = U$. Aquí $d = 4$ y $H(4) \approx 2{,}08$, así que la garantía del teorema siguiente permite hasta 4,16 y 3 la cumple.

Teorema 3.9 (Johnson, 1974; Chvátal, 1979)**.** Sean J^* una cobertura de tamaño mínimo y J_g la que devuelve el voraz. Con $d = \max_j |S_j|$ y $H(d) = 1 + \frac{1}{2} + \cdots + \frac{1}{d}$,

$$|J_g| \leq H(d)\,|J^*|.$$

Como $d \leq r$, también $|J_g| \leq H_r\,|J^*|$ con $H_r \leq \ln r + 1$.

Idea de la prueba (Chvátal). Cuando el voraz elige una interacción que cubre g categorías nuevas, reparte un precio de $1/g$ a cada una de ellas. La suma de todos los precios es exactamente $|J_g|$.

Sea $S \in J^*$ con $|S| = s$. Se ordenan sus categorías $e_1, \ldots, e_s$ según el momento en que el voraz las cubre. Cuando se cubre e_q, todavía faltan por cubrir al menos $s - q + 1$ categorías de S (la propia e_q y las posteriores). Entonces S misma habría dado una ganancia de al menos $s - q + 1$, y el voraz eligió una con ganancia mayor o igual. Por lo tanto e_q paga a lo sumo $\frac{1}{s-q+1}$, y las categorías de S pagan en total a lo sumo

$$\frac{1}{s} + \frac{1}{s-1} + \cdots + \frac{1}{1} = H(s) \leq H(d).$$

Como las interacciones de J^* cubren todo U, sumando sobre ellas se obtiene $|J_g| \leq |J^*|\,H(d)$.

Observación 3.10. El voraz no garantiza el óptimo: solo garantiza estar a un factor $H(d)$ de él, como muestra el Ejemplo 3.8.

3.4 Análisis de eficiencia

Se cuentan las instrucciones del Algoritmo 2 con la misma convención de la Sección 2.4: la línea L_k cuesta C_k por ejecución y un ciclo **para** evalúa su encabezado una vez más que el número de vueltas. Sea p el número de vueltas del ciclo **mientras** (interacciones elegidas), y para la vuelta $t = 1, \ldots, p$:

- $u_t = n - t + 1$ es el número de interacciones aún no elegidas;
- $s_t = \sum_{j \text{ no elegida}} |S_j| \leq s$, con $s = \sum_{j=1}^{n} |S_j| \leq n \cdot r$;
- $\sigma_t = |S_{mejor_t}| \leq d$, con $d = \max_j |S_j| \leq r$.

Línea	Instrucción	Costo	Veces
1	**para** $k \leftarrow 1$ **hasta** r	C_1	$r+1$
2	$cubierta[k] \leftarrow$ falso	C_2	r
3	**para** $j \leftarrow 1$ **hasta** n	C_3	$n+1$
4	$elegida[j] \leftarrow$ falso	C_4	n
5	$faltan \leftarrow r$; $J \leftarrow \emptyset$	C_5	1
6	**mientras** $faltan > 0$	C_6	$p+1$
7	$mejor \leftarrow$ nulo; $g^* \leftarrow 0$	C_7	p
8	**para** $j \leftarrow 1$ **hasta** n	C_8	$\sum_{t=1}^{p}(n+1) = p(n+1)$
9	**si** $elegida[j]$	C_9	$\sum_{t=1}^{p} n = p\,n$
10	$g \leftarrow 0$	C_{10}	$\sum_{t=1}^{p} u_t = pn - \frac{p(p-1)}{2} \le p\,n$
11	**para cada** $k \in S_j$	C_{11}	$\sum_{t=1}^{p}(s_t + u_t) \le p\,(s+n)$
12	**si no** $cubierta[k]$: $g \leftarrow g+1$	C_{12}	$\sum_{t=1}^{p} s_t \le p\,s$
13	**si** $g > g^*$: actualizar $mejor$	C_{13}	$\sum_{t=1}^{p} u_t \le p\,n$
14	marcar $elegida[mejor]$; $J \leftarrow J \cup \{mejor\}$	C_{14}	p
15	**para cada** $k \in S_{mejor}$	C_{15}	$\sum_{t=1}^{p}(\sigma_t + 1) \le p\,(d+1)$
16	**si no** $cubierta[k]$	C_{16}	$\sum_{t=1}^{p} \sigma_t \le p\,d$
17	$cubierta[k] \leftarrow$ verdadero; $faltan \leftarrow faltan - 1$	C_{17}	r (cada categoría se cubre una vez)
18	**devolver** J	C_{18}	1

Sumando costo por veces y agrupando:

$$T(n,r) \;\le\; \alpha\,p\,s + \beta\,p\,n + \gamma\,p\,d + \delta\,p + \varepsilon\,r + \zeta\,n + \eta,$$

con $\alpha = C_{11} + C_{12}$, $\beta = C_8 + C_9 + C_{10} + C_{11} + C_{13}$, $\gamma = C_{15} + C_{16}$,
$\delta = C_6 + C_7 + C_8 + C_{14} + C_{15}$, $\varepsilon = C_1 + C_2 + C_{17}$, $\zeta = C_3 + C_4$ y $\eta = C_1 + C_3 + C_5 + C_6 + C_{18}$.

Cuántas vueltas hay. Por el Lema 3.7, $p \le \min(n, r)$. Con $s \le n\,r$ y $d \le r$:

$$T(n,r) \;\le\; \alpha\,\min(n,r)\,n\,r + \beta\,\min(n,r)\,n + \gamma\,\min(n,r)\,r + \delta\,\min(n,r) + \varepsilon\,r + \zeta\,n + \eta.$$

El término dominante es el primero, y por lo tanto

$$\boxed{T(n,r) = O\big(n \cdot r \cdot \min(n,r)\big) \subseteq O(n \cdot r^2)}$$

En términos del tamaño total s de los conjuntos, la cota es $O\big(\min(n,r)\,(n+s)\big)$. Es una cota superior; no se afirma que sea ajustada para toda entrada.

Mejor caso. La primera interacción cubre todo: $p = 1$ y $T(n,r) = \Theta(n + r + s)$, porque aun así se revisan todos los candidatos en la primera vuelta. La comprobación de factibilidad cuesta $O(r+s)$ y queda absorbida.

Memoria. Los arreglos *cubierta* y *elegida* ocupan $O(r+n)$, además de la propia entrada.

4 Resultados

4.1 Cómo se obtuvieron

Todos los resultados salen de ejecutar, desde la raíz del repositorio y sin editar el código,

```
python ejecutar_casos.py    y    python ejecutar_casos.py casos_propios
                        --salida resultados_propios
```

con la versión `entrega1-1.1` y Python 3.13.18. Para cada caso el programa guarda la salida, el tiempo y la traza en `resultados/caso_XX_<nombre>/`, y corre verificadores independientes que releen el archivo original y recalculan todo desde cero. El tiempo es el del algoritmo solo (sin leer el archivo ni escribir resultados), con la mediana de varias repeticiones; depende de la máquina, así que sirve para comparar entre casos y no como valor absoluto.

4.2 Caso de la profesora

El archivo `entrada_G3_mochila_20_nodos.json` tiene 20 etapas en cadena, $B = 72$ y 12 categorías de riesgo. La Tabla 2 resume la salida y el tiempo; las Tablas 3 y 4 son las trazas.

Parte 1: aceleración	
Método	programación dinámica (cadena)
Etapas / presupuesto B	20 / 72
Etapas aceleradas	E02, E03, E04, E06, E08, E12
Costo usado	72 de 72
Duración	501 → 446
Ahorro	55
Óptimo confirmado por enumeración de los 2^{20} subconjuntos	sí
Tiempo (mediana de 15 repeticiones)	0,0732 ms
Parte 2: auditoría	
Candidatos	20 etapas (una por etapa del flujo), $r = 12$ categorías
Interacciones elegidas (en orden)	E01, E16, E06, E03
Tamaño de la cobertura	4
Óptimo por enumeración	4
Cota $H(d) \cdot \lvert J^* \rvert$ con $d = 4$	8,33
Tiempo (mediana de 15 repeticiones)	0,0211 ms

Tabla 2: Salida y tiempo de la solución base sobre el caso de la profesora.

La solución acelera seis etapas y gasta exactamente los 72 del presupuesto. El verificador independiente probó los 2^{20} subconjuntos y confirmó que ninguno logra una duración menor que 446.

El voraz cubre las 12 categorías con 4 interacciones. La enumeración de todas las combinaciones confirma que 4 es el mínimo, así que en este caso el voraz es óptimo.

4.3 Casos propios

Los casos de la profesora no cubren flujos con rutas paralelas, cadenas grandes ni instancias infactibles. Por eso se generaron once casos propios con semilla fija (`generar_casos.py`); la Tabla 5 los resume. En la columna P1, DP es programación dinámica y B&B es ramificación y acotamiento; "Nodos" son los nodos explorados por B&B.

Tres observaciones sobre la tabla. (i) En `02_voraz_no_optimo` el voraz elige 3 interacciones y el óptimo es 2 (Ejemplo 3.8); se cumple la cota $H(d)\,|J^*|$ del Teorema 3.9. En `11_reduccion_grafo_5_vertices` se corre la instancia que produce la reducción del Ejemplo 3.6: el voraz elige b, c y d, y la enumeración confirma que el mínimo es 3. (ii) En

Etapa	b antes	$DP[i][b]$	Decisión	Costo	Ahorro	b después
E20	72	55	no	0	0	72
E19	72	55	no	0	0	72
E18	72	55	no	0	0	72
E17	72	55	no	0	0	72
E16	72	55	no	0	0	72
E15	72	55	no	0	0	72
E14	72	55	no	0	0	72
E13	72	55	no	0	0	72
E12	72	55	acelerar	9	7	63
E11	63	48	no	0	0	63
E10	63	48	no	0	0	63
E09	63	48	no	0	0	63
E08	63	48	acelerar	17	13	46
E07	46	35	no	0	0	46
E06	46	35	acelerar	13	10	33
E05	33	25	no	0	0	33
E04	33	25	acelerar	16	12	17
E03	17	13	acelerar	5	4	12
E02	12	9	acelerar	12	9	0
E01	0	0	no	0	0	0

Tabla 3: Traza de la reconstrucción del DP en el caso de la profesora, de la última etapa a la primera. La columna b antes es el presupuesto que queda al llegar a la etapa; $DP[i][b]$ es el ahorro óptimo con las etapas hasta esa y ese presupuesto. La tabla completa de 21×73 celdas está en `parte1_traza.json`.

`03_diamante` acelerar la rama corta no reduce la duración, y el solver general acelera solo la rama crítica. (iii) En `10_paralelo_40_ancho8` el solver explora unos 50 mil nodos, frente a unos cientos en los casos de 20 y 30 etapas, y tarda alrededor de un segundo. Esa es la parte del costo que puede crecer de forma exponencial en el peor caso.

4.4 Escala del DP

El análisis de la Sección 2.4 dice que el tiempo crece como $n \cdot B$. La Tabla 6 compara el tiempo medido con el número de celdas de la tabla en las cadenas disponibles.

Los nanosegundos por celda se mantienen del mismo orden al pasar de unas mil a unos dos millones y medio de celdas. Esto es coherente con $\Theta(nB)$; no es una demostración, solo una comprobación empírica del análisis.

4.5 Supuestos y límites de la evidencia

- **Candidatos a auditar.** La profesora confirmó que cada etapa es un candidato a auditar con su propia lista de riesgos y que se usa la auditoría completa. El programa usa exactamente eso: lee solo `riesgos_cubiertos_auditoria_completa` e ignora el campo `variaciones_auditoria` del archivo.
- **Casos futuros.** La profesora confirmó que habrá más casos en el mismo formato, posiblemente con más etapas, estaciones o caminos. Por eso el lector valida el archivo con mensajes de error claros, el orden de las etapas sale de `flujo` y no del orden de la

Vuelta	Elegida	Ganancia g^*	Categorías nuevas	Cubiertas	Faltan
1	E01	4	R01, R02, R03, R10	4	8
2	E16	4	R04, R05, R09, R12	8	4
3	E06	3	R07, R08, R11	11	1
4	E03	1	R06	12	0

Tabla 4: Traza del voraz en el caso de la profesora: orden de elección, categorías nuevas por vuelta y cobertura acumulada de las $r = 12$ categorías.

Caso	n	Aristas	B	P1	Nodos	Duración	ms P1	$\|J\|$	ms P2	Verif.
`01_ejemplo_informe_n4`	4	3	7	DP	–	36→26	0,009	2	0,004	sí
`02_voraz_no_optimo`	3	2	6	DP	–	18→13	0,005	3	0,005	sí
`03_diamante`	4	4	5	B&B	1	14→8	0,013	2	0,004	sí
`04_rutas_paralelas_20`	20	26	72	B&B	261	144→102	2,706	5	0,024	sí
`05_rutas_paralelas_30`	30	42	100	B&B	413	264→197	6,708	7	0,036	sí
`06_cadena_200`	200	199	1500	DP	–	4594→3270	17,730	7	0,252	sí
`07_cadena_500`	500	499	5000	DP	–	10861→7167	171,615	12	1,209	sí
`08_infactible`	3	2	4	DP	–	18→15	0,005	infactible	–	sí
`09_presupuesto_cero`	3	2	0	DP	–	18→18	0,006	3	0,005	sí
`10_paralelo_40_ancho8`	40	128	150	B&B	49737	151→120	1075,331	7	0,050	sí
`11_reduccion_grafo_5_vertices`	5	4	5	DP	–	22→16	0,013	3	0,006	sí

Tabla 5: Casos propios (tiempos: mediana de 5 repeticiones). Todos pasan los verificadores independientes. El caso `08_infactible` deja una categoría sin cubrir y la Parte 2 lo reporta como infactible, nombrando la categoría.

lista, y la Parte 1 resuelve flujos con rutas paralelas. Se desconoce el tamaño máximo de esos casos.

- **Alcance.** Esta entrega no incluye experimentos de escalamiento sistemáticos, las variantes ni la heurística; quedan para la segunda entrega.

5 Conclusiones

Ambas partes del problema son NP-difíciles: la Parte 1 por reducción desde la mochila 0/1 y la Parte 2 por reducción desde la cobertura de vértices; las dos reducciones son polinomiales y se probaron en los dos sentidos. Sin embargo, esa dificultad se comporta distinto en cada parte.

En la Parte 1 el algoritmo de programación dinámica es exacto y cuesta $\Theta(n \cdot B)$. Es pseudopolinomial: rápido mientras B sea pequeño en valor, e inviable si B crece exponencialmente respecto de su escritura. Cuando el flujo tiene rutas paralelas la estructura de mochila se pierde y se usa un método exacto exponencial en el peor caso, que en los casos probados terminó en tiempos pequeños.

En la Parte 2 el voraz cuesta $O(n \cdot r \cdot \min(n, r))$ y no garantiza el óptimo, pero queda a un factor $H(d) \leq \ln r + 1$ de él. En el caso de la profesora coincide con el óptimo, y en un caso propio queda a una interacción de distancia.

Quedan pendientes, para la segunda entrega, los experimentos de escalamiento, las variantes y una heurística para flujos grandes.

Caso	n	B	Celdas $(n+1)(B+1)$	ms	ns por celda
`caso de la profesora`	20	72	1 533	0 073	47 7
`06_cadena_200`	200	1500	301 701	17 730	58 8
`07_cadena_500`	500	5000	2 505 501	171 615	68 5

Tabla 6: Tiempo del DP frente al número de celdas $(n+1)(B+1)$.

Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2022). *Introduction to Algorithms* (4.ª ed.). MIT Press.
- Chvátal, V. (1979). A greedy heuristic for the set-covering problem. *Mathematics of Operations Research*, 4(3), 233–235.
- Garey, M. R. y Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman.
- Johnson, D. S. (1974). Approximation algorithms for combinatorial problems. *Journal of Computer and System Sciences*, 9(3), 256–278.
- Karp, R. M. (1972). Reducibility among combinatorial problems. En *Complexity of Computer Computations*, 85–103. Plenum Press.